LSFL: A Lightweight and Secure Federated Learning Scheme for Edge Computing

Zhuangzhuang Zhang, Libing Wu, Chuanguo Ma, Jianxin Li, *Senior Member, IEEE*, Jing Wang, Qian Wang, *Senior Member, IEEE*, and Shui Yu, *Senior Member, IEEE*

***Abstract*— Nowadays, many edge computing service providers expect to leverage the computational power and data of edge nodes to improve their models without transmitting data. Federated learning facilitates collaborative training of global models among distributed edge nodes without sharing their training data. Unfortunately, existing privacy-preserving federated learning applied to this scenario still faces three challenges: 1) It typically employs complex cryptographic algorithms, which results in excessive training overhead; 2) It cannot guarantee Byzantine robustness while preserving data privacy; and 3) Edge nodes have limited computing power and may drop out frequently. As a result, the privacy-preserving federated learning cannot be effectively applied to edge computing scenarios. Therefore, we propose a lightweight and secure federated learning scheme LSFL, which combines the features of privacy-preserving and Byzantine-Robustness. Specifically, we design the Lightweight Two-Server Secure Aggregation protocol, which utilizes two servers to enable secure Byzantine robustness and model aggregation. This scheme protects data privacy and prevents Byzantine nodes from influencing model aggregation. We implement and evaluate LSFL in a LAN environment, and the experiment results show that LSFL meets fidelity, security, and efficiency design goals, and maintains model accuracy compared to the popular FedAvg scheme.**

***Index Terms*— Federated learning, edge computing, privacy-preserving, Byzantine-robustness, data privacy.**

Manuscript received 30 November 2021; revised 23 August 2022; accepted 31 October 2022. Date of publication 14 November 2022; date of current version 7 December 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101100; in part by the National Natural Science Foundation of China under Grant U20A20177, Grant U22B2022, and Grant 62202197; in part by the Key Research and Development Plan of Hubei Province under Grant 2021BAA025; in part by the Industry-University-Research Innovation Fund for Chinese Universities under Grant 2021FNA04004; and in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ) under Grant GML-KF-22-07. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Thorsten Strufe. *(Corresponding author: Libing Wu.)*

Zhuangzhuang Zhang and Libing Wu are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen 511442, China (e-mail: zhzhuangzhuang@whu.edu.cn; wu@whu.edu.cn).

Chuanguo Ma and Qian Wang are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: flinty@whu.edu.cn; qianwang@whu.edu.cn).

Jianxin Li is with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia (e-mail: jianxin.li@deakin.edu.au).

Jing Wang is with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: cswjing@hust.edu.cn).

Shui Yu is with the School of Computer Science, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: shui.yu@uts.edu.au).

Digital Object Identifier 10.1109/TIFS.2022.3221899

I. Introduction

IN RECENT years, with the development of Internet of Things (IoT) technology, a large number of IoT devices have the ability to provide intelligent and customized computing services. A report by Statista [1] estimates that the number of IoT connected devices will reach 30.9 billion in 2025. These computing-capable IoT devices also generate a large amount of data. Nowadays, how to utilize these devices and data has become a hot topic for both industry and academia. Many service providers (e.g., Apple, Google, and Tesla) are willing to use their own edge nodes for machine learning training (See Figure 1). For example, an autonomous vehicle service provider has a trained autonomous driving model (e.g., Traffic Sign Recognition, Blind Spot Monitoring, and Pedestrian Detection System), it wants to use the vehicles sold for joint training. For consumers, they may not want to disclose their data privacy information to the service provider. To resolve this problem, the service provider can leverage the *Federated learning* (FL) [2], [3] techniques to jointly learn the model with the consumer, without sharing the consumer's local raw training data. In FL, there are multiple clients (e.g., IoT devices, and smartphones) and a service provider called aggregator [4]. Unlike traditional machine learning, the client (also called participant) and server of FL can jointly perform the training task. Concretely, each participant holds a local dataset and trains a model locally, and then exchanges only intermediate parameters with the aggregator, instead of privacy-sensitive training data. After then, aggregator aggregates the model parameters of different participants. In this way, the service provider performs model training without transmitting massive training data.

FL has been widely used in edge computing scenarios due to the benefits mentioned above, such as Internet of Vehicles (IoV) [5], [6], smart home [7], [8], smartphone [2], [9], and Internet of Medical Things (IoMT) [10], [11]. However, FL still has two issues that need to be mitigated. The first issue is the privacy leakage of local model updates. For example, Melis et al. [12] demonstrate that the intermediate gradients may leak *unintended* information about participants' training data. In [13], Hitaj et al. show that *distributed, federated, or decentralized deep learning* approach do not protect the training set of honest participants. In other words, adversary can learn privacy information from local model updates uploaded by participants. To address aforementioned privacy implications, Differential Privacy (DP)-based [14], [15], Homomorphic Encryption (HE)-based [16], [17], and Secure

1556-6021 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

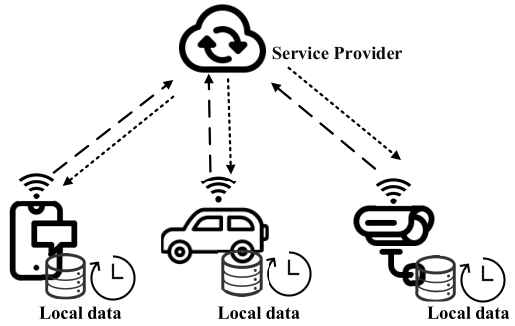


Fig. 1. An example scenario of machine learning for edge computing. In this scenario, both the deployed devices and service providers prefer to obtain better quality AI models.

Multi-Party Computation (SMC)-based [18], [19], [20] schemes have been proposed. Each of these schemes has benefits and drawbacks. The DP-based scheme has a low computational overhead but reduces global model accuracy due to the noise it introduces. In contrast, the SMC and HE-based schemes have almost no impact on the global model accuracy, but their computational overhead is high because of their reliance on complex cryptographic primitives. In addition, edge nodes have limited computing resources to support complex cryptographic algorithms. Thus, how to enable a lightweight privacy-preserving FL is the first problem to be solved.

The second issue is that privacy-preserving FL is still vulnerable to Byzantine attacks. Byzantine participants may compromise the global model via data poisoning attacks [21], [22] or local model poisoning attacks [23], [24], [25]. To resist Byzantine attacks, various Byzantine robust FL [4], [26], [27], [28], [29] schemes have been proposed. Although these schemes can remove malicious local model updates by comparing the similarity or analyzing the data distribution of local model updates, these schemes lack privacy preservation for the participants. Intuitively, these schemes require direct access to the local model updates instead of the protected updates. In practice, edge nodes generally do not want to share local model updates directly, but edge nodes are extremely vulnerable to corruption. Thus, how to conduct Byzantine attack defense while ensuring the privacy of edge nodes is the second problem to be solved.

In recent years, some works [30], [31], [32], [33] has been proposed to solve both of these problems. For example, Dong et al. proposed FLOD [31], which addresses the issue that FLTrust [4] does not preserve the privacy of the participants. FLOD utilizes two non-colluding servers to preserve the privacy of the aggregation phase and uses Hamming distance instead of cosine similarity in FLTrust for Byzantine attack defense. However, FLOD uses HE and MPC, which introduces a certain computational overhead and requires participants to conduct encryption and decryption operations offline. This is not in line with the resource-constrained situation of edge nodes. Similarly, Ma et al. [33] proposed the differentially private Byzantine robust FL scheme (DPBFL), which combines differential privacy and RSA aggregation rules to conduct Byzantine attack defense while preserving privacy. However, DPBFL introduces a noise, which has a negative impact on the global model accuracy.

Obviously, the previous work is still not efficient to resist Byzantine attacks, while ensuring high efficiency and security. Our goal is to mitigate these two problems. More precisely, we intend to solve the following problem: *"How to make a FL scheme robust against Byzantine adversaries while protecting the privacy of individual participants through a computationally lightweight approach?*". Therefore, we proposes a new lightweight secure FL scheme called LSFL, which combines the functional properties of privacy-preserving and Byzantine-robustness. It can be applied in the scenario of collaborative training using edge nodes. Specifically, LSFL contains two honest-but-curious servers without collusion and the participant. The two non-colluding honest-but-curious servers are responsible for secure model aggregation and secure Byzantine robustness, while the participant is responsible for local model training. It is worth noting that the each training round results of the participants are not uploaded to the servers in their complete form, but are uploaded to two servers using a secret sharing method. Hence, neither server can know the real training results of the participants, and cannot deduce the participants' private information. In this way, we can achieve privacy preserving of local model parameters without using complex cryptographic protocols. Compared with previous works [30], [31], [32], [33], it reduces the computational overhead while having no effect on the model's accuracy. In addition, our LSFL supports the dynamic drop out of participants during the training phase.

Contributions. The main contributions of this paper are summarized as follows:

- We propose LSFL, a lightweight crowdsourced FL scheme that aids edge node vendors in improving service quality. It preserves the consumer's privacy and allows the consumer to drop out during the training phase.
- We propose a *Lightweight Byzantine-Robustness Two-Server Secure Aggregation* protocol. It can achieve both secure aggregation and secure Byzantine-robustness. Specifically, it preserves the privacy information of the participants in the Byzantine robustness and model aggregation phases.
- We implement a prototype of LSFL and evaluated their performance in terms of training accuracy, training time, and Byzantine robustness. Our results show that LSFL can achieve the fidelity, efficiency and security design goals.

Organization. The rest of this paper is organized as follows. In Section II, we present a description of the related work in this paper. In Section III, we describe the system overview, the threat model, and the design goals. In Section IV, we describe this scheme in detail. In Section V, we analyze the security of this scheme. Then, we perform experimental evaluation in Section VI. Finally, we conclude the paper in Section VII.

II. Related Work

In this section, we briefly discuss the key research work related to this paper.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

FL as a privacy-preserving distributed learning paradigm has been widely used in edge computing scenarios, such as Internet of Vehicles (IoV) [5], [6], smart home [7], [8], smartphone [2], [9], and Internet of Medical Things (IoMT) [10], [11]. In particular, to effectively leverage the computing resources of smartphones, Google first proposed a FL scheme (called FedAvg) [2] that uses a server to aggregate the model parameters of the training participants, thus enabling efficient collaborative training. Based on this, Hard et al. [9] proposed a FL method for mobile phone keyboard prediction, and this work demonstrated the feasibility and benefits of model training that can be achieved without directly leaking user data privacy to the server. Zhao et al. [7] proposed a FL system to assist home appliance manufacturers to train model based on consumers' data. However, they combine the use of differential privacy and blockchain to preserve consumer privacy, which may affect the model accuracy and training time seriously.

DP preserves the privacy of the data holder by adding random noise to the raw data or model parameters so that the data cannot be recovered. For example, Abadi et al. [14] proposed a deep learning scheme based on DP that uses Gaussian noise to obfuscate gradients and experiments demonstrated that deep neural networks with non-convex objectives can be trained, under a modest privacy budget. Compared to [14], [39] protect the whole dataset although it employs a DP-based FL scheme. In addition, [39] can achieve dynamic adaptation of differential privacy during distributed training. The work [40] also proposed DP-based SGD algorithms and analyzed their performance bounds, and the analysis showed that they are related to privacy level and dataset size. All the above work [7], [14], [39], [40] based on DP struggles to find a trade-off between model accuracy and privacy level.

Considering the difficulty of this trade-off, several works have proposed secure multiparty computing (SMC) [18], [19], [20] and homomorphic encryption (HE) [17], [34], [41]. SMC using multiparty computing protocols (MPC) can enable collaborative computing without revealing information about the parties. Chaudhari et al. [18] proposed Trident, an efficient 4PC framework for privacy-preserving machine learning. To support more parties, Sav et al. [19] proposed POSEIDON, a privacy-preserving federated neural network learning system. It adopts a secure lattice-based multiparty computing protocol to preserve the confidentiality of data and models, under N parties. In addition, to adapt to stronger adversary capabilities, Lehmkuhl et al. [20] proposed MUSE, an efficient two-party secure inference protocol resilient to malicious clients. Compared to SMC, HE enables machine learning to complete training without decrypting data, thus preserving user privacy. Xu et al. [17] proposed VerifyNet, a secure and verifiable FL scheme by combining homomorphic encryption and secret sharing techniques. Li et al. [41] proposed a secure and privacy-preserving distributed deep learning scheme for fog-cloud computing that encrypts intermediate gradient information using the Paillier encryption algorithm and verifies user identity using threshold signature techniques. Giacomelli et al. [34] proposed a scheme for training a linear regression model that uses only linear HE. Also, there are some methods that combine two or three of the above techniques to achieve better performance [35], [42]. However, the computational and communication overheads of these schemes are too high.

Nevertheless, participants in FL may be malicious. To address these issues, several works [4], [26], [27], [28], [29] have proposed Byzantine-robust secure aggregation method. The primary defense strategy to protect the model against Byzantine user attacks is to compare model updates from different users and remove outliers. For example, in [29], Blanchard et al. proposed Krum and Multi-Krum, which can perform model training with Byzantine user participation. In addition, Wu et al. [28] point that SGD is suffering from a considerable approximation error (noise) associated with the stochastic gradient. To overcome this challenge, Wu et al. [28] proposed Byrd-SAGA, a new stochastic average gradient algorithm based on [43] that enables defense against Byzantine users. However, all of these schemes require access to the participant's plaintext data. In other words, they do not protect participant data privacy. This motivates us to achieve both privacy-preserving and Byzantine robustness of the participants.

Similarly, several works have been proposed in recent years to achieve privacy-preserving and Byzantine-robust FL. For example, Dong et al. proposed FLOD [31], a private Byzantine-robust FL scheme. FLOD achieves privacy-preserving Byzantine-robust FL by improving the privacy of FLTrust using HE and two-party computation techniques. However, FLOD requires HE operations on the participants, which causes a large computational overhead. This is inconsistent with the resource-constrained nature of edge nodes. So et al. [30] also proposed a Byzantine-resistant secure aggregation framework (BREA) for secure FL. Nevertheless, BREA requires multiple communications between the server and the participants to complete the secure aggregation, which incurs a huge communication overhead. In this paper, we design an efficient privacy-preserving and Byzantine-robust FL scheme for the same purpose. Moreover, our scheme does not incur an additional computational burden on the participants since it does not use complex cryptographic algorithms. A detailed comparison of the LSFL with existing schemes is described in Table I. The results indicate that our scheme can achieve efficient Byzantine robustness and privacy preservation. Meanwhile, there exist some works which focus on the problems that edge nodes may face during their participation in FL training. For example, Wang et al. [44], [45] proposed algorithms to solve the problem of insufficient resources when edge nodes participate in FL. In this paper, we only consider how to utilize the computing power and data of the edge nodes in a secure and efficient way.

III. Problem Statement

In this section, we first present the system overview, and then describe the threat model and the design goals.

A. System Overview

Figure 2 presents the overview of LSFL. The LSFL comprises three entity types, namely: Third Party (TP), Service Provider (SP) and Data Providers (DPs). TP and SP jointly manage the FL tasks.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON BETWEEN OUR WORK AND PREVIOUS WORKS

Solution	Techniques		Threat Model		Privacy Guarantee	Server	Byzantine Robustness	Efficiency
	Data privacy	Aggregation Rules	Aggregator	Participant				
Solution [16]	HE	FedAvg	HbC	HbC	●	Single	○	○
Solution [34]	HE	FedAvg	HbC	HbC	●	Single	○	○
Krum [29]	–	Krum	Honest	Malicious	○	Single	●	●
Hybridalpha [35]	FE	FedAvg	HbC	HbC	●	Single	○	○
SecureFL [32]	HE	FLTrust	HbC	Malicious	●	Dual	●	◐
Solution [36]	MPC	FedAvg	HbC	HbC	●	Single	○	○
BREA [30]	MPC	Krum	HbC	Malicious	●	Single	●	○
FLTrust [4]	–	Cosine Similarity	Honest	Malicious	○	Single	●	◐
PEFL [37]	HE	Pearson correlation coefficient	HbC	Malicious	●	Single	●	○
PPFL [38]	TEE	FedAvg	HbC	HbC	●	Single	○	○
DPBFL [33]	DP	RSA	HbC	Malicious	●	Single	●	◐
FLOD [31]	MPC	Sign-SGD & FLTrust	HbC	Malicious	●	Dual	●	○
LSFL (our work)	2PC	Secure Byzantine Robustness & Secure Fairness Guarantee	HbC	Malicious	●	Dual	●	●

* The symbol ● indicates YES, ○ indicates NO, ◐ indicates partially YES, – indicates that this item is not available.
† "HbC" indicates Honest-but-Curious, "Single" and "Dual" indicates the number of servers.

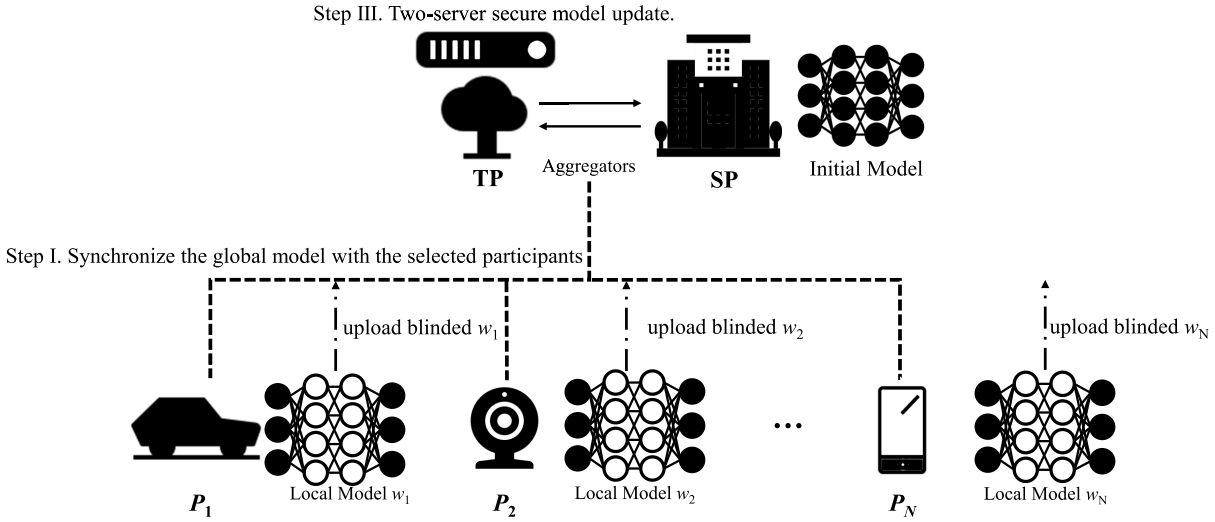


Fig. 2. System Overview.

- **TP:** The TA is a third party in the system, which is responsible for the selection of eligible participants (i.e., DPs) used for model aggregation. More precisely, the TA excludes suspicious Byzantine nodes from the DPs, thus ensuring that the SP can get the correct model.
- **SP:** The SP is an edge computing service provider in the system, which can be considered as a smartphone provider (e.g., Apple, Huawei) or an autonomous driving service provider (Tesla, Ford). It maintains the initial model and expects to optimize the model using the computational resources and data in the DPs. In simple terms, it intends to retrain the model on the data owned by the DPs. In particular, the SP is responsible for the aggregation of the models i.e. the updating of the models and maintaining the latest models.
- **DPs:** The DP is an edge computing node. It collects a large amount of data through its own sensors and is willing to participate in joint training to get a better model. We also assume that it is a non-trustworthy participant with certain computational power that may contribute inappropriate intermediate results in the training process.

B. Threat Model

We consider the following threat model:

1) Honest-But-Curious SP and TP: SP and TP follow the protocol correctly but may try to learn additional information. In other words, SP and TP attempt to obtain some private information from the participants while following the protocol, so that they can use this information to infer the model privacy

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

of the participants (i.e., edge node). Although SP and TP do not have the incentive to disrupt the FL training process, they may be driven by commercial interests to obtain the private information of edge nodes.

2) Byzantine DPs: In practice, edge nodes (i.e., DPs) are vulnerable to attacks due to their close proximity to the user side. Therefore, we do not guarantee that the edge nodes can execute all the training tasks correctly. Further, we assume that the edge nodes follow the protocol and are curious about the information returned by the SP. However, they may use their information to send arbitrary local model updates. Similar to the prior works [27], [29], [37], we also assume that most of the DPs are not malicious, which is also consistent with the real scenario. More specifically, we consider two types of Byzantine nodes:

- Free-riders. Free-riders are lazy participants. They do not want or have the ability to share their local model updates and only want access to the trained model by accessing the FL training process. They typically upload random local model updates [46]. Here, we use Gaussian attack [25] to simulate free-riders. This attack injects the noise of Gaussian distribution into the local model update.
- Malicious nodes. For malicious nodes, it usually launches attacks with a purpose. Here, we assume that it launch a sign-flipping attack. Sign-flipping attack [47] is a untargeted poisoning attack that flips the element-wise signs of the local model updates.

C. Design Goals

Our aim is to design a lightweight FL solution that can address the current model training needs of edge computing service providers. The scheme can protect participants' privacy while protecting them from Byzantine participants. In other words, the scheme needs to enable an effective combination of privacy preserving and Byzantine robustness. In this paper, we use FedAvg as a baseline to discuss fidelity, security, and efficiency. FedAvg is a standard FL scheme with no security measures. The global model in this scheme is the average of local model updates from different users. Our scheme's aggregation rules are more comparable to FedAvg, but it also includes a number of security measures for privacy-preserving and Byzantine robustness. Specifically, the FL scheme we designed needs to meet the following design goals:

- **Fidelity.** The accuracy of the global model cannot be sacrificed by the scheme when there is no attack. Here, we use FedAvg as a baseline for comparison. In other words, the scheme is able to train a global model that is as accurate as FedAvg when there is no attack.
- **Security.** The scheme should satisfy two security goals: (i) It should protect the privacy of the participants in the training process, (ii) It should guarantee the correct model training in the presence of Byzantine participants. In particular, with the involvement of Byzantine nodes, this scheme should learn a model that is as accurate as FedAvg.
- **Efficiency.** Due to the limited computational resources of the edge nodes themselves, the scheme cannot incur additional computational and communication overhead, especially for the participants. In particular, with the involvement of Byzantine nodes, our scheme cannot increase the workload of the participants compared to FedAvg. In addition, our scheme cannot add too much time overhead to the overall training process, compared to FedAvg. In simple terms, our scheme can only introduce a limited time overhead.

IV. System Design

In this section, we describe the LSFL in detail. The LSFL can ensure that (i) the honest-but-curious aggregator cannot learn additional knowledge from the participants except for output of the participant, (ii) the Byzantine nodes cannot learn the parameters of other honest participants, (iii) the Byzantine nodes have no impact on model accuracy. In the rest of this, we represent the structure of our system and its working process.

A. High-Level Description of LSFL

A high-level description of the system is in algorithm 1. For each round, DPs, TP, and SP jointly perform the FL training. For simplicity, we use $\mathcal{P}$ to denote DPs, $\mathcal{P} = \{P_1, P_2, \ldots, P_{\mathcal{N}}\}$. $\mathcal{P}$ want to collaboratively train a machine learning model locally without sharing local data with any other entities in the system.

Specifically, FL iteratively performs the following three steps (illustrated in Figure 2):

1) Step I: Synchronize the Global Model With the Selected Participants: In this step, SP sends latest global model w^{global} to the selected participants $\mathbb{P} \in \mathcal{P}$. Participants $\mathbb{P}$ save the model for later training.

2) Step II: Training Local Models: After receiving the global model, each selected participant trains a local model by fine-tuning the global model using its local training dataset. Formally, the P_i try to solves the optimization problem $\min_{w_i} \mathcal{L}(D_i, w_i)$, where $\mathcal{L}(D_i, w_i)$ is the empirical loss on the training dataset D_i, w_i is the participants P_i's local model. Then, each participant splits its local model update to obtain $w_i^{(1)}$ and $w_i^{(2)}$, and send them to SP and TP respectively.

3) Step III: Two-Server Secure Model Update: To secure model updates and preserve data privacy, the SP and TP jointly compute global model $\mathcal{W}$ by *secure aggregating* the secret-shared local model updates according to *aggregation rule*. The *aggregation rule* consist of *Secure Byzantine Robustness*, *Secure Fairness Guarantee*, and *Secure Model Aggregation*. This allows the SP and TP to (i) know nothing about the update parameters of any participant, (ii) avoid Byzantine attacks to reduce global model accuracy or even corrupt the model, (iii) encourage participants to take part in training and ensure the fairness of different participants.

B. Private Local Models Training

We assume that each participant P_i maintains a local dataset D_i, where $i = \{1, 2, 3 \ldots n\}$, $D = \bigcup_{i=1}^{n} D_i$. Aggregators TP and SP keep part of the global model parameters, respectively.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 A High-Level Description of the System

Input: DPs $P_i \in \mathcal{P}$, maximum number of communication round R, global model w^{global}, maximum number of epochs E, P_i's money γ_i
Output: Latest model $\mathcal{W}$

Initialize R, w^{global}, E
$\mathbb{P} \leftarrow \mathcal{P}$
for communication round $r = 1$ to R **do**
  // Step I: Synchronize the global model with the selected participants
  SP sends global model w^{global} to the participants $\mathbb{P}$
  // Step II: Training local models
  for each participant $P_i \in \mathbb{P}$ **do**
    for epoch $e = 1$ to E **do**
      Run SGD independently on the local dataset
    end for
    Upload the blinded w to the aggregators SP and TP
    if $\gamma_i < 0$ **then**
      $\mathbb{P}.remove(P_i)$
    end if
  end for
  // Step III: Two-server secure model update
  The aggregators choose to accept w by using *secure Byzantine robustness* and *secure fairness guarantee* mechanism
  SP and TP update each participant's money γ_i through the *secure fairness guarantee*
  The aggregators perform model updates to obtain $\mathcal{W}_{new}$ by the *secure two-server model aggregation* protocol
end for
$\mathcal{W} \leftarrow \mathcal{W}_{new}$
return Latest model $\mathcal{W}$

Only after the collaborative computing of TP and SP, the SP can reconstruct the correct global model. Among them, SP not only maintains the model parameters but is also responsible for the secure aggregation. In other words, participants can only receive the latest model provided by SP for training, and the training results are sent to TP and SP respectively. This process is detailed in Algorithm 2.

More specifically, the SP maintains the initial model to encourage the FL participants to conduct collaborative training. Firstly, aggregator SP sends latest model $w^{(global)}$ to the selected participants $\mathbb{P}$ (Notably, only in the first iteration of FL, $\mathbb{P}$ and $\mathcal{P}$ are identical.). Secondly, each participant $P_i \in \mathbb{P}$ trains the local model using standard SGD algorithm. In one round of training, they iterate multiple epochs in the local dataset $D_{(i)}$. Specifically, the participant overwrites his local model with the received global model parameters and computes

$$g_i = \nabla \mathcal{L}(w, D_{(i)}), \tag{1}$$

g_i reflects the best direction in which the model parameters change. Then, the participant computes

$$w_i \leftarrow w_i - \eta g_i. \tag{2}$$

Algorithm 2 Private Local Model Training

Input: learning rate η, aggregators TP, SP, local model w, maximum number of epochs E, loss function $\mathcal{L}(w, D)$, Gaussian noise variance g^2.
Output: secret-shared local model of current round w

Initialize the model parameter w^0.
for $i = 1$ to E **do**
  Take a random batch D_b from samples D.
  // Gradient descent
  $w_i \leftarrow w_{i-1} - \eta \nabla \mathcal{L}(w, D_b)$
end for
// Split private w_i into additive secret shares.
Generate random noise $\zeta_i \sim N(0, g^2)$
$\langle w_i \rangle = \{w_i^{(1)}, w_i^{(2)}\}$, where $w_i^{(1)} = w - \zeta_i$ and $w_i^{(2)} = w + \zeta_i$
Send $w_i^{(1)}$, $w_i^{(2)}$ to TP and SP, respectively.

Compare to most FL scheme, the purpose we perform model parameter calculations here is (i) to reduce the burden of the aggregator, (ii) use the computing resources of the participants (e.g., smartphone).

Besides, to improve the security of model parameters and disperse the security risks of the aggregator, we employ a two-server architecture for secure aggregation. In this architecture, each participant P_i obtains $w_i^{(1)}$, $w_i^{(2)}$ and sends them to the SP, TP. Specifically, each participant $P_i \in \mathbb{P}$ splits its private value w_i into two additive secret values. Formally, private value w_i can be represented as

$$\begin{aligned} w_i &= \frac{1}{2}(w_i^{(1)} + w_i^{(2)}) \\ &= \frac{1}{2}(w_i + \zeta_i + w_i - \zeta_i), \end{aligned} \tag{3}$$

where ζ_i is a Gaussian noise satisfying the $N(0, g^2)$ distribution. Here, we set $g = 20$. It should also be noted that each participant generates a different noise. This prevents the Byzantine participant from inferring the noise of other participants. After then, participant P_i sends $w_i^{(1)}$, $w_i^{(2)}$ to TP, SP respectively. Subsequently, the aggregator will begin the *Two-Server Secure Model Update*, which is explained in the next section.

C. Two-Server Secure Model Update

After receiving the secret-shared model parameters from each participant P_i, the aggregators start to update the global model and distribute the updated model parameters to each participant. Generally, the current single-aggregator FL aggregation scheme largely depends on the single-aggregator, and the single-aggregator is vulnerable to attacks from external adversaries. And then, model privacy might be compromised by data infringements, hacking, leaks. To avoid single point of failure of single-aggregator FL scheme, we propose a *Lightweight Byzantine-Robustness Two-Server Secure Aggregation* protocol.

As shown in Algorithm 3, our aggregation protocol mainly consists of three components: *secure Byzantine robustness*, *secure fairness guarantee*, and *secure model aggregation*.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Lightweight Byzantine-Robustness Two-Server Secure Aggregation Protocol

Input: aggregators TP, SP, secret-shared local model of current round $w_i^{(1)}$, $(w_i)^{(2)}$, participants $\mathcal{P} = \{P_1, P_2, \ldots, P_i, P_\mathcal{N}\}$, penalty factor φ, money γ, reward factor ς, cost σ, number of participants $\mathcal{N}$, selected participants $\mathbb{P}$

Output: global model parameter $\mathcal{W}$

Initialize: φ, γ, ς, σ, k
$\forall\ P_i$, TP receive $w_i^{(1)}$ and SP receive $w_i^{(2)}$
// Secure Byzantine Robustness
$z_1 \leftarrow \sum_1^\mathcal{N} w_i^{(1)}$
$z_2 \leftarrow \sum_1^\mathcal{N} w_i^{(2)}$
TP sends its share of z_1 to SP.
SP recovery $\bar{w}$ via $\bar{w} \leftarrow \frac{1}{2\mathcal{N}}(z_1 + z_2)$.
SP computes $\frac{1}{2}w_i^{(2)} - \bar{w}$ and send it to TP.
for all $w_i \in w$ **do**
  TP computes $d_i \leftarrow \frac{1}{2}w_i^{(1)} + \frac{1}{2}w_i^{(2)} - \bar{w}$
  $d(w_i) \leftarrow |d_i|$
end for
$d' \leftarrow \{d(w_1), d(w_2), \ldots, d(w_\mathcal{N})\}$
TP gets the median m of d'
for all $w_i \in w$ **do**
  TP computes $d_i'' = |d(w_i) - m|$
end for
TP choose the top k participants with the smallest similarity $\mathscr{P} \leftarrow \{P_1, P_2, \ldots, P_k\}$.
// Secure Fairness Guarantee
while $i \leq \mathcal{N}$ **do**
  if $P_i \in \mathscr{P}$ **then**
    $\gamma_i \leftarrow \gamma_i + \varsigma$
    $\gamma_i \leftarrow \gamma_i - \sigma$
    $i \leftarrow i + 1$
  else
    $\gamma_i \leftarrow \gamma_i - \varphi$
    $\gamma_i \leftarrow \gamma_i - \sigma$
    $i \leftarrow i + 1$
  end if
  if $\gamma_i < 0$ **then**
    $\mathbb{P}.remove(P_i)$
  end if
end while
// Secure Model Aggregation
TP computes $\mathcal{W}^{(1)} \leftarrow \frac{1}{2k}\sum_{i=1}^k w_i^{(1)},\ P_i \in \mathscr{P}$
SP computes $\mathcal{W}^{(2)} \leftarrow \frac{1}{2k}\sum_{i=1}^k w_i^{(2)},\ P_i \in \mathscr{P}$
TP sends $\mathcal{W}^{(1)}$ to SP
SP computes $\mathcal{W} \leftarrow \mathcal{W}^{(1)} + \mathcal{W}^{(2)}$
SP reveals $\mathcal{W}$ to participants $\mathbb{P}$, and updates the global model.

1) Secure Byzantine Robustness: To mitigate the impact of lazy or Byzantine participants on the global model, we should to evaluate the model parameters uploaded by the participants and select the appropriate model parameters. Therefore, we design an efficient *k-nearest* weight selection method that is resilient to Byzantine participants. Concretely, it mainly includes the following steps:

First, TP, SP compute $\sum_{i=1}^\mathcal{N} w_i^{(1)}$ and $\sum_{i=1}^\mathcal{N} w_i^{(2)}$ respectively, then TP sends its $\sum_{i=1}^\mathcal{N} w_i^{(1)}$ to SP;

Second, SP gets the *average model* $\bar{w}$ by computing

$$\bar{w} = \frac{1}{2\mathcal{N}}\left(\sum_{i=1}^{\mathcal{N}} w_i^{(1)} + \sum_{i=1}^{\mathcal{N}} w_i^{(2)}\right); \tag{4}$$

Third, SP computes $\frac{1}{2}w_i^{(2)} - \bar{w}$ and sends it to TA;

Fourth, TP computes the distances d_i between different participant models and the *average model* by computing

$$d_i = \frac{1}{2}w_i^{(1)} + \frac{1}{2}w_i^{(2)} - \bar{w} \tag{5}$$

to obtain $d = \{d_1, d_2, \ldots, d_\mathcal{N}\}$;

Fifth, since d_i is a vector, TP transforms it into $d(w_i) = \|d_i\|_2$ to obtain $d' = \{d(w_1), d(w_2), \ldots, d(w_\mathcal{N})\}$, and then takes the median value m of d', where $\|\cdot\|_2$ denotes the l_2-norm. However, since the average model can have outliers, such as Byzantine participants contributing extremely large model update parameters, it can cause the average model to deviate from the normal range. Therefore, in the last step, TP reveals the distance between w_i and m by computing $d_i'' = |d(w_i) - m|$, and use it as a criterion for judging similarity. According to the above steps, we can obtain the deviation degree of different participants from the average model. Based on d'', TA obtain the selected participants $\mathbb{P}$ via picks top *k-nearest* participants' model update w_i, then use *Secure Model Aggregation* to update the global model.

2) Secure Fairness Guarantee: To protect the fairness of the participants and reduce the impact of Byzantine participants on the global model, we propose a reward and penalty mechanism based on the behavior of the participants. The mechanism's primary goal is to ensure that bad participants (i.e., participants that do not contribute to the global model) are punished or even removed, and that benign participants (i.e., participants that contribute positively to the global model) are rewarded. As a result, the mechanism's challenge is to distinguish between benign and bad participants. To overcome this challenge, we use the contribution of a participant to determine whether it is a benign participant. A participant is regarded as benign if it adds to the global model aggregation; otherwise, it is regarded as a bad participant. In other words, the fundamental principle of the mechanism is to determine whether a participant P_i contributes to the convergence of the global model $\mathcal{W}$. If the participant P_i has a positive influence on the global model $\mathcal{W}$, TP give him a reward ς, if not, TP penalizes the participant P_i. In addition, the server allocates the same amount of money to each participant before the training starts, and the participant must pay for each round of training. The evaluation of whether a participant contributes to the global model is based on whether the participant was selected by the aggregation server in the previous Byzantine Robustness phase. If it is not chosen, we consider that it has only acquired the global model and has not contributed to global model aggregation, so we punish it. Obviously, if a participant continues to launch attacks, it will quickly deplete its money. It cannot participate in FL training if

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

it runs out of money. In this way, we can remove the Byzantine participant and thus reduce the impact of the Byzantine participant on the global model aggregation. Specifically, first, the aggregation server allocates an equal amount of money γ to each participant; second, TP verifies whether the participants P_i have sufficient money, and after determining that they have sufficient money, TP distributes the global model $\mathcal{W}$ and deducts the corresponding costs σ; third, the participant performs local training and uploads the training results w_i after receiving the global model parameters $\mathcal{W}$; finally, TP uses the result of the participant selection in step *Secure Byzantine Robustness* to reward the selected participants, and punish those who are not selected. By repeating the aforementioned steps, participants who contribute weakly to the global model cannot take part in the subsequent training after their money is exhausted, thus we succeeded in achieving the purpose of ensuring fairness.

3) Secure Model Aggregation: To enable lightweight data privacy preservation, we propose a two-server secure aggregation protocol. The protocol uses a secret sharing mechanism to keep the participant P_i's model updates w_i in TP and SP separately. In other words, TP holds the participants P_i's model update $w_i^{(1)}$, where $i \in [1, \mathcal{N}]$. Correspondingly, SP holds P_i's model update $w_i^{(2)}$. Next, aggregators perform *Secure Model Aggregation*. Specifically, TP, SP first locally compute $\mathcal{W}^{(1)} = \frac{1}{2k}\sum_{i=1}^{k} w_i^{(1)}$, $\mathcal{W}^{(2)} = \frac{1}{2k}\sum_{1}^{k} w_i^{(2)}$ respectively, where $P_i \in \mathbb{P}$; second, TP sends its share $\mathcal{W}^{(1)}$ to SP; then SP obtains global model $\mathcal{W}$ by $\mathcal{W} = \mathcal{W}^{(1)} + \mathcal{W}^{(2)}$; finally, SP updates the global model with $\mathcal{W}$. By this step, the SP gets the training results for the current round.

V. Security Analysis

In this section, we analyze the security of LSFL and demonstrate that our scheme satisfies the security goals.

Theorem 1: SP, TP and Byzantine DPs can know nothing about the sensitive information of honest-but-curious DPs.

Proof: In this scheme, each participant DP P_i $(i \in [1, N])$ splits its local model update w_i to obtain $w_i^{(1)}$ and $w_i^{(2)}$, and sends them to aggregators TP and SP, respectively. Furthermore, SP sends $\frac{1}{2}w_i^{(2)} - \bar{w}$ to TP during the *Secure Byzantine Robustness* phase. In other words, for aggregator TP, it only knows the knowledge $\{w_i^{(1)}\}_{i=1}^{N}$ received from participants P_i and the $\frac{1}{2}w_i^{(2)} - \bar{w}$ received from SP. Correspondingly, aggregator SP only knows the $\{w_i^{(1)}\}$, $\frac{1}{2}w_i^{(2)} - \bar{w}$ and $\sum_{i=1}^{N} w_i^{(1)}$, where N is number of participant DPs involved in the aggregation. And since TP and SP are non-colluding servers, they cannot know the exact value of w_i for P_i. Thereby, honest-but-curious aggregators TP and SP cannot use the received weight information w_i to infer the model and data privacy of the participants.

In addition, each participant P_i only knows the local model w_i and the global model $\mathcal{W}$. For the honest-but-curious participant P_i, it is clear that its not possible to know the w_j of other participant P_j. For $M(M < N)$ Byzantine DPs, we assume that they conspire to obtain sensitive information. Thus, they get $\mathcal{W}_h \leftarrow \mathcal{W} - \mathcal{W}_{adv}$, where $\mathcal{W}_h$ is the sum of the local models $w_i, i \in [1, N-M]$ of honest-but-curious DPs and $\mathcal{W}_{adv}$ is the sum of the local models $w_i, i \in [1, M]$ of Byzantine DPs. Obviously, we can see that Byzantine DPs can infer honest-but-curious DPs for w_i when $M = N - 1$. However, this assumption is unrealistic, as such an FL task is meaningless. Also, we mainly consider that most DPs are not Byzantine nodes to fit the realistic scenario, similar to previous work [27], [29], [37]. Hence, they cannot obtain any sensitive information about the local model of honest DPs.

Therefore, neither the non-colluding SP and TP nor the Byzantine DPs can know any sensitive information about the honest-but-curious DPs. □

Theorem 2: Our Two-server Secure Model Update is secure as long as SP and DP do not collude.

Proof: During the process of Two-server Secure Model Update, the view of SP in real-world is defined as $\text{REAL}_{\text{SP}} = (w^{(2)}, \bar{w}, z_1, z_2, \mathcal{W}^{(1)}, \mathcal{W}^{(2)}, \mathcal{W})$, where $w^{(2)} = \{w_i^{(2)}\}_{i=1}^{N}$ are secret shares of corresponding $\{w_i\}_{i=1}^{N}$, $\mathcal{W}^{(1)} = \frac{1}{2k}\sum_{k}^{i=1} w_i^{(1)}$, and $\mathcal{W} = \mathcal{W}^{(1)} + \mathcal{W}^{(2)}$ is the global model. Simulate it in ideal-world, the view of SP is $\text{IDEAL}_{\text{SP}} = (w^{(2)}, \bar{w}', z_1', z_2, \mathcal{W}^{(1)'}, \mathcal{W}^{(2)'}, \mathcal{W}')$. Because SP only knows $\mathcal{W}^{(1)}$ and $\mathcal{W}$, its unable to infer the $w^{(1)}$. Therefore, we can conclude that these two views are computationally indistinguishable.

For TP, the view of TP in the real-world is defined as $\text{REAL}_{\text{TP}} = (w^{(1)}, z_2, \frac{1}{2}w_i^{(2)} - \bar{w}, d', \mathcal{W}^{(1)})$. To simulate it in ideal-world, the simulator can simply return $(w^{(1)}, z_2', \frac{1}{2}w_i^{(2)} - \bar{w}', d', \mathcal{W}^{(1)})$, where z_2', $\frac{1}{2}w_i^{(2)} - \bar{w}'$ are chosen from $\mathcal{R}^d$ at random. In other words, in the ideal world, there exists a view, i.e., $\text{IDEAL}_{\text{TP}} = (w^{(1)}, z_2', \frac{1}{2}w_i^{(2)} - \bar{w}', d', \mathcal{W}^{(1)})$. Obviously, these two views are also computationally indistinguishable.

Therefore, we prove that our Two-server Secure Model Update is secure as long as SP and DP do not collude. We can guarantee that the adversary (corrupted SP or TP) learns nothing except the information that the SP and TP should know. □

Theorem 3: LSFL is robust against Byzantine DPs, as long as most DPs are honest-but-curious.

Proof: In this scheme, we select the k best local models for aggregation. Specifically, we use $d(w_i) = \|w_i - \bar{w}\|_2$ to calculate the similarity between the local model (i.e., w_i) and the baseline model (i.e., $\bar{w}$), and then use $d_i'' = |d(w_i) - m|$ to select the most similar local model, where $\|\cdot\|_2$ is l_2-*norm*, and m is the median of $d(w_i)(i \in [1, N])$. According to the above steps, the k DPs (i.e., $P_i(i \in [1, k])$) with the smallest d_i'' are selected. In other words, if P_i is honest-but-curious, then $d_i'' \leq \delta$. It is worth noting that we assume that most DPs are honest-but-curious, so here we set $k = 50\%$. Let w_a denote the local model from Byzantine participants and w_h denote the local model update from honest participants. Here, we assume that all Byzantine DPs can collude to launch attacks together, i.e., launch specific attacks at the same time to affect global model updates. For Byzantine DP, if he wants the attack to succeed, he must make sure that equations $|\|w_a - \bar{w}\|_2 - m| \leq \delta$ and $\|w_a\|_2 \gg \|w_h\|_2$ hold, i.e., let the poisoned local model w_a be aggregated into the global model with sufficient negative

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

impact. But honest-but-curious DPs are in the majority, hence $min(\|w_h - \bar{w}\|_2) < m \leq max(\|w_h - \bar{w}\|_2)$. In addition, when the poisoned local model w_a is too large, $|\|w_a - \bar{w}\|_2 - m| > \delta$. Obviously, the equations of Byzantine DP cannot be hold.

As a result, our LSFL is robust against Byzantine DPs, as long as most DPs are honest-but-curious. □

VI. Evaluation

In this section, we perform the detailed evaluation of our LSFL. In particular, we have evaluated LSFL for fidelity, security, and efficiency.

A. Experimental Setup

1) Datasets: We evaluated LSFL using two datasets from different domains, which is MNIST [48] and CIFAR-10 [49].

- **MNIST dataset:** MNIST is a standardized dataset consisting of greyscale images (28×28) of the digits 0–9. The dataset has a training set of 60,000 examples, and a test set of 10,000 examples [48]. We use the 2-layer CNN and the simple MLP to conduct experiments on the IID dataset and the non-IID dataset, respectively. The CNN and MLP architecture specified in FedAvg [2].
- **CIFAR-10 dataset:** CIFAR-10 is a standardized dataset consisting of color images (32×32) divided into 10 classes. The dataset consists of 50,000 training examples and 10,000 testing examples. In our experiments, we only consider the IID case. And, our experiments use the 2-layer CNN and the simple MLP architecture specified in FedAvg [2].

2) Data Splits: We used two ways (IID and Non-IID) to split the dataset for each DP (i.e., participant): 1) For IID, we shuffled the MNIST and CIFAR-10 datasets and distributed them equally to each DP; 2) For non-IID, we divided the MNIST dataset into 200 shards of size 300 each and distributed two shards to each DP.

3) Baselines: We use the following five aggregation rules as baselines.

- **FedAvg [2]:** FedAvg is a standard FL scheme with no security measures. It averages the local model returned by the participants and updates it as the global model.
- **SignSGD [50]:** The aggregation server takes signs for the updated gradients, then sums them and signs the summation result as a model update. It is worth noting that the process of summing the results collected from the participants at the server is actually a process of computing the majority vote.
- **DP-SignSGD [51]:** DP-SignSGD combines the Gaussian mechanism with SIGNSGD. Specifically, the participants send the signed gradients with differential privacy to the aggregation server, which then averages them and takes the sign to update the global model.
- **Krum [29]:** Krum chooses a model from among several local models that is similar to the others as the global model, i.e., the gradient that is similar to the majority of the gradients is chosen as the aggregated gradient for global model update.
- **Median [27]:** Median takes the median of the local model updates of the different participants and uses it as the global model update.

4) Hyper-Parameters: In our experimental setup, we tested six cases: MNIST MLP Non-IID, MNIST MLP IID, MNIST CNN Non-IID, MNIST CNN IID, CIFAR MLP IID, and CIFAR CNN IID. For local training, we set the learning rate as 0.001, the local epochs as 5, the SGD momentum as 0.5, and the local batch size as 10. For FL, we set the ratio of selected participants k to 70% in each round. In addition, each participant is initialized with the same parameter $\mathcal{W}_0$, subsequent local model training is carried out on its own dataset.

5) Adversaries: We consider two types of adversaries (i.e., Byzantine DPs): free-riders and malicious DPs. In each experiment, we test the performance of LSFL under both types of attacks. For free-riders, they upload Gaussian noise, where each element follows Gaussian distribution $\sim N(0, g^2)$ and we set $g^2 = 30$. For malicious DPs, they conduct sign-flipping attacks. Specifically, a malicious participant P_i computes the true value $\hat{v}_i$ and sends $v_i = q\hat{v}_i$ to the server, where q is a negative constant. In all experiments, we set $q = -1$. Briefly, Byzantine DPs contain free-riders and malicious DPs.

B. Experimental Results

To evaluate the three design goals of our LSFL, we perform three experiments. They mainly include the evaluation of the fidelity, security, and efficiency. In the following, we describe these three experiments in detail.

1) Fidelity Evaluation: To evaluate the fidelity goal of this scheme, we compare FedAvg [2] and LSFL. Notably, there is no attack. In this experiment, we set both the number of participants and communication round as 100. Figure 3 shows the convergence of the LSFL and the FedAvg [2] over the MNIST and CIFAR-10 dataset. We can observe that our scheme does not have any impact on the model in terms of convergence. Its convergence is highly similar to the FedAvg. This profile does not change on either the MNIST or the CIFAR-10 dataset.

At the same time, to test the effect of this scheme on the model's accuracy, we compare the accuracy of the FedAvg and ours LSFL. Figure 4 present the accuracy of the proposed scheme and the FedAvg over the MNIST (IID and Non-IID) and IID CIFAR-10 dataset. We can observe from Figure 4 that the accuracy of our approach is almost identical to that of the FedAvg on the MNIST and CIFAR-10 datasets. In other words, our approach has no effect on the accuracy of the model. This is also because our scheme does not introduce noise but uses only secret sharing, which does not have an impact on the training accuracy of the model. It demonstrate that our scheme enables the fidelity goal.

Moreover, we also evaluated the impact of the total number of participants on FL model accuracy. Figure 5 shows the accuracy of our proposed LSFL under different global model and dataset, when the total number of participants $\mathcal{N}$ increases from 20 to 100. We can observe that the number of participants has a positive effect on accuracy, but the effect becomes

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

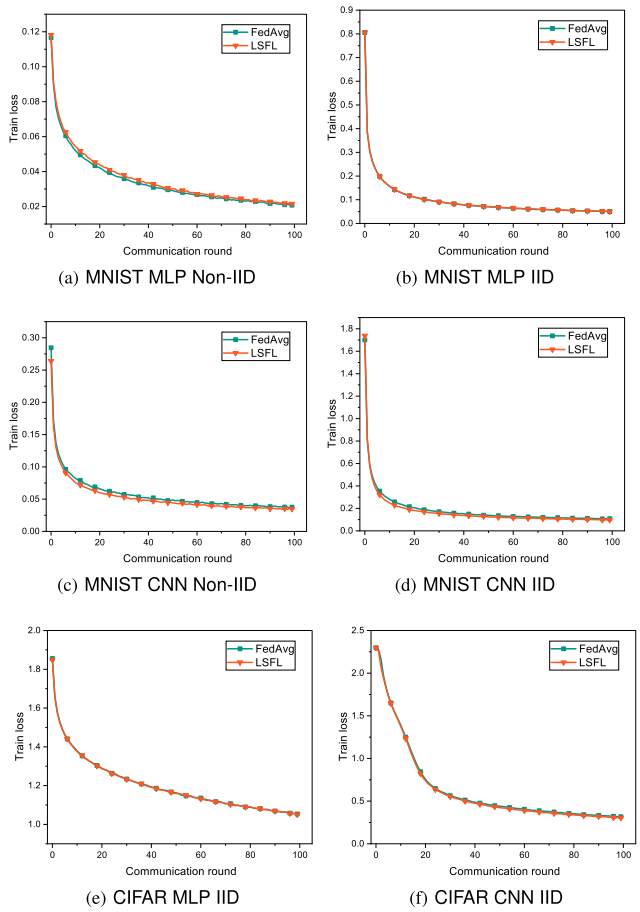


Fig. 3. Convergence of the proposed LSFL and the FedAvg over the MNIST (IID and Non-IID) and IID CIFAR-10 dataset.

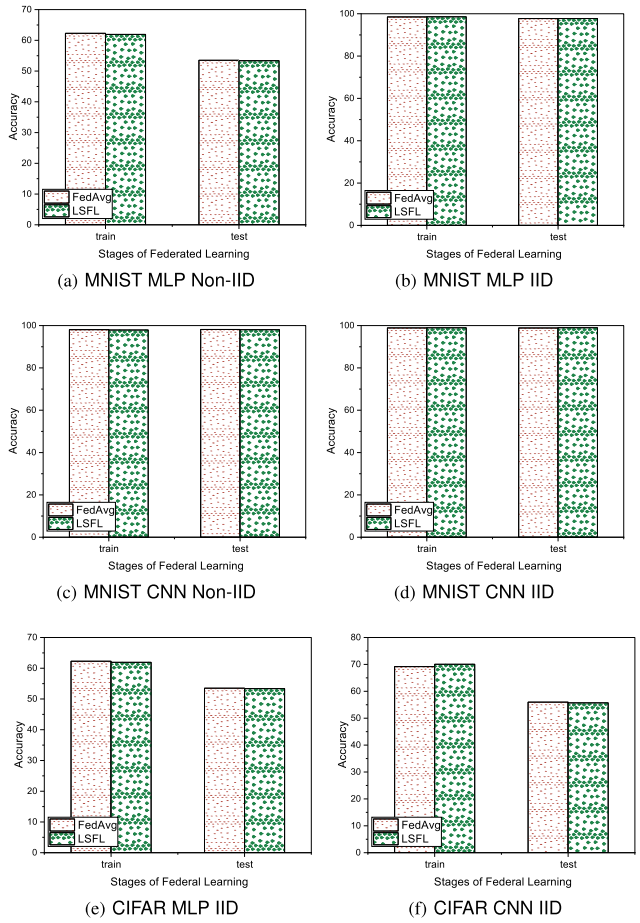


Fig. 4. Accuracy of the LSFL and the FedAvg on the MNIST(IID and Non-IID) and IID CIFAR-10 dataset.

minimal after a certain threshold. This is because the improvement of model accuracy requires a suitable contribution of the participants to the global model convergence, and the number of participants cannot form an effective positive impact on the model convergence when an effective contribution cannot be provided.

2) Security Evaluation: To evaluate the security goal of this scheme, we first compared and tested the performance of our LSFL with different numbers of Byzantine DPs. Specifically, we tested the accuracy of LSFL in the case of 10% Byzantine DPs, 20% Byzantine DPs, and 30% Byzantine DPs on the MNIST and CIFAR-10 dataset, and compared it with FedAvg (0% and 10% Byzantine DPs). Byzantine DPs include free-riders and malicious DPs. In this experiment, we set both the number of DPs and communication round as 100. The experimental results are shown in Table II and Table III. We can observe that our LSFL (under attack) is almost as accurate as FedAvg (no attack). This illustrates that our LSFL has outstanding Byzantine resilience and can enable the same model prediction accuracy as FedAvg in the presence of Byzantine DPs. This is because *Lightweight Two-Server Secure Aggregation* protocol selects k correct participants and excludes Byzantine participants. Meanwhile, we can also observe that free-riders have a greater impact on FedAvg than malicious DPs. This is because free-riders randomly generate

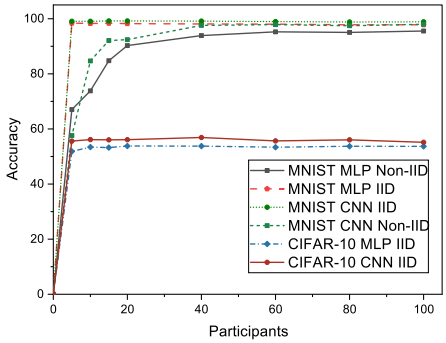


Fig. 5. Impact of the total number of participants on LSFL accuracy under different cases (MNIST MLP Non-IID, MNIST MLP IID, MNIST CNN Non-IID, MNIST CNN IID, CIFAR MLP IID, and CIFAR CNN IID).

Gaussian noise, which has a greater impact on the global model. In contrast, the malicious DP's sign-flipping attack sends the opposite value of the real data to the aggregation server in order to slow down the global model's convergence. Limited by the magnitude of these values, their impact on the global model is rather less than that of the free-riders. In other words, if we reduce the noise generated by free-riders, their impact on the global model will be reduced. In addition, this scheme does not introduce additional factors that can affect

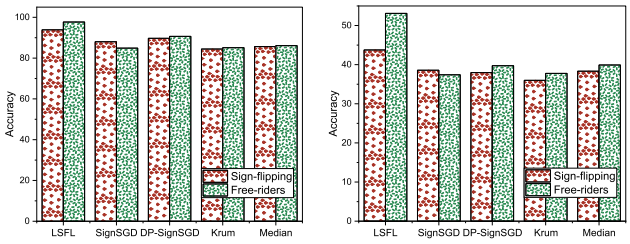
Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE TEST ACCURACY OF LSFL AND FEDAVG UNDER FREE-RIDER DPS

	MNIST MLP Non-IID	MNIST MLP IID	MNIST CNN Non-IID	MNIST CNN IID	CIFAR MLP IID	CIFAR CNN IID
FedAvg (0% Byzantine DPs)	95.69%	97.73%	97.89%	98.82%	53.68%	55.09%
FedAvg (10% Byzantine DPs)	37.66%	62.39%	11.88%	8.78%	19.66%	10.17%
LSFL (10% Byzantine DPs)	94.31%	97.58%	96.97%	98.92%	53.27%	54.61%
LSFL (20% Byzantine DPs)	95.58%	97.64%	97.78%	98.87%	53.08%	52.22%
LSFL (30% Byzantine DPs)	95.07%	97.66%	97.88%	98.84%	53.12%	53.03%

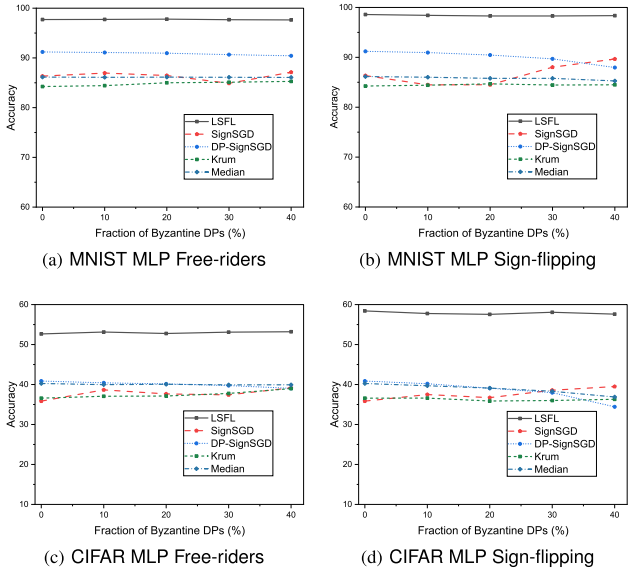
TABLE III
THE TEST ACCURACY OF LSFL AND FEDAVG UNDER MALICIOUS DPS

	MNIST MLP Non-IID	MNIST MLP IID	MNIST CNN Non-IID	MNIST CNN IID	CIFAR MLP IID	CIFAR CNN IID
FedAvg (0% Byzantine DPs)	95.69%	97.73%	97.89%	98.82%	53.68%	55.09%
FedAvg (10% Byzantine DPs)	72.58%	90.20%	73.20%	93.74%	37.48%	10.00%
LSFL (10% Byzantine DPs)	91.13%	97.91%	95.01%	98.78%	59.83%	67.68%
LSFL (20% Byzantine DPs)	93.01%	98.09%	95.88%	98.86%	60.11%	69.13%
LSFL (30% Byzantine DPs)	92.90%	98.10%	96.39%	98.71%	61.40%	67.20%



(a) MNIST MLP (b) CIFAR MLP

Fig. 6. The accuracy of different FL methods under free-rider and sign-flipping attacks.



(a) MNIST MLP Free-riders (b) MNIST MLP Sign-flipping

(c) CIFAR MLP Free-riders (d) CIFAR MLP Sign-flipping

Fig. 7. Impact of the fraction of DPs on the accuracy of different FL methods under different attacks (i.e., free-riders and sign-flipping).

the accuracy of the model such as noise. In other words, our scheme achieves the security design goal.

In addition, to further evaluate the security of LSFL, we compared it with existing schemes (i.e., SignSGD, DP-SignSGD, Krum, Median). Specifically, we tested the accuracy of our LSFL, SignSGD, DP-SignSGD, Krum, and Median when suffering from free-rider and sign-flipping attacks on the MNIST and CIFAR-10 datasets. In this experiment, we set the number of DPs and the number of communication rounds to 20, the percentage of Byzantine users to 30%, and used MLP as the global model. The experimental results are shown in Figure 6. From Figure 6, we can observe that our LSFL has better prediction accuracy when it suffers from Byzantine attacks. In particular, the advantage of LSFL is more obvious when Byzantine DPs are free-riders. This illustrates that our scheme has better Byzantine robustness. This is also because the key step of our scheme is to use the median as a baseline to select the best participant, which is robust to outliers such as Gaussian noise.

Meanwhile, the fraction of Byzantine DPs also affects the performance of different schemes. To evaluate the impact of Byzantine DPs' fraction, we tested the prediction accuracy of different schemes when Byzantine DPs' fraction ranged from 0% to 40%. In this experiment, we set the number of DPs and the number of communication rounds to 20 and use MLP as the global model. The experimental results are shown in Figure 7. Figure 7 shows that LSFL has the highest accuracy when compared to other schemes. Since we assume that most DPs are not malicious, we only tested Byzantine DPs within 50% and set k to 50%. This indicates that our scheme is more robust to Byzantine attacks. Also, we can see that the accuracy of LSFL does not decrease with the increase of Byzantine DPs.

Since in the step *Secure Model Aggregation*, we set the value of k to determine the number of participants to be selected. To further evaluate the impact of this value on the LSFL, we tested the accuracy of the model with 100 participants (with 30% Byzantine participants) for k values of 10%, 30%, 50%, and 70%, respectively. In this experiment, we set the communication round as 100. The experimental results are shown in Figure 8. We can observe that the number of selected participants has a positive impact on the testing accuracy of the model in the absence of Byzantine participants. However, after the contribution of the participants to the global model reaches a certain level, their positive impact on the model is negligible.

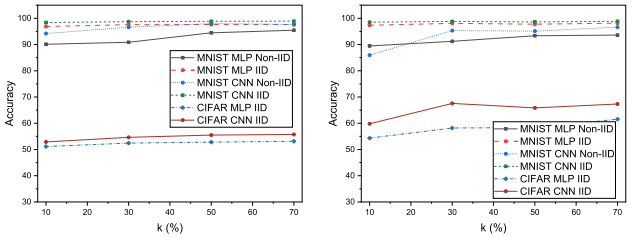
Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

COMPARISON OF COMPUTATION AND COMMUNICATION OVERHEAD BETWEEN OUR LSFL AND [37], [52]

Phase	Our LSFL	Solution [37]	Solution [52]
Comp.	$\mathcal{O}(T_{tr}) + \mathcal{O}(mT_{ns}) + \mathcal{O}(2mT_{add})$	$\mathcal{O}(T_{tr}) + \mathcal{O}(mT_{enc})$	$\mathcal{O}(T_{tr}) + \mathcal{O}(mT_{enc}) + \mathcal{O}(mT_{mul})$
Comm.	$\mathcal{O}(2m\|w\|)$	$\mathcal{O}(m\|w_{enc}\|)$	$\mathcal{O}(m\|w_{enc}\|)$

* The m denotes the dimension for uploading intermediate parameters, T_{tr} denotes the time overhead of local model training, and T_{ns} denotes the time overhead of noise generation for a number, T_{enc} denotes the time required for homomorphic encryption, and T_{add}, T_{mul} denotes the time for addition and multiplication, respectively.



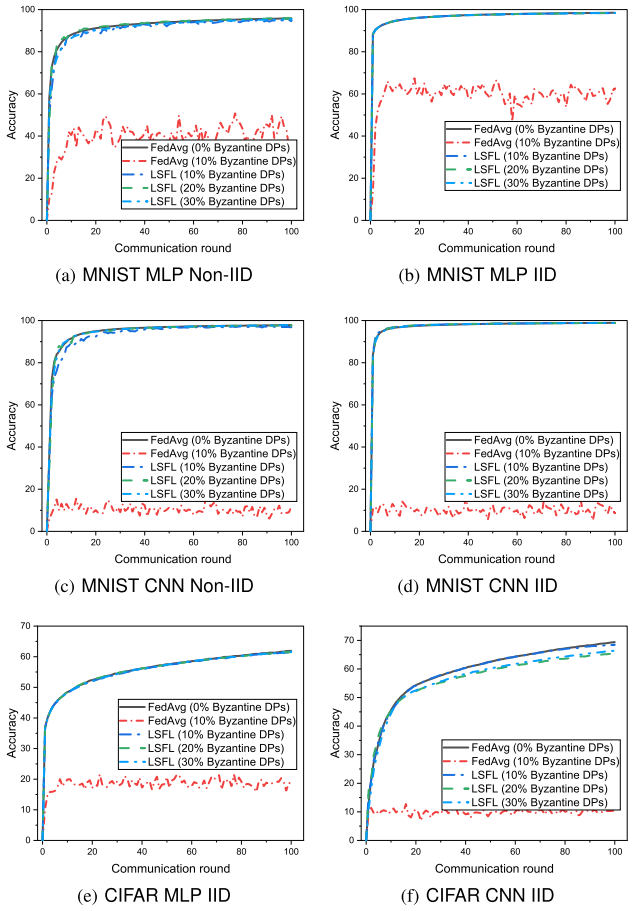
(a) Free-riders (b) Sign-flipping

Fig. 8. Impact of the k value on LSFL accuracy under different cases (MNIST MLP Non-IID, MNIST MLP IID, MNIST CNN Non-IID, MNIST CNN IID, CIFAR MLP IID, and CIFAR CNN IID).

This is because with the exclusion of Byzantine participants, as the number of participants increases, the aggregator can receive more training results and thus can obtain a better global model.

3) Efficiency Evaluation: To evaluate the efficiency goal of this scheme, we tested the accuracy of FedAvg and LSFL for different numbers of Byzantine DPs. Specifically, we tested the accuracy of LSFL in the case of 10% Byzantine DPs, 20% Byzantine DPs, and 30% Byzantine DPs on the MNIST and CIFAR-10 dataset, and compared it with FedAvg (0% and 10% Byzantine DPs). In this experiment, we set both the number of DPs and communication round as 100. The experimental results are shown in Figure 9 and 10. Figures 9 and 10 show the accuracy of our LSFL and FedAvg when Byzantine DPs are free-riders and malicious participants who launch sign-flipping attacks, respectively. For all cases of LSFL in Figure 9 and 10, we set k as 70. This is also because we assume that most of the participants DPs are honest, and that these participants honestly contribute useful intermediate parameters for model training. We can observe that the convergence rate of LSFL accuracy is the same as that of FedAvg under 0% malicious DPs in six cases. In particular, LSFL can achieve almost the same accuracy as FedAvg with the participation of different malicious DPs for the same number of communication rounds, while providing Byzantine robustness and privacy preserving. This illustrates that our LSFL has almost no effect on the speed of model convergence, which means that LSFL also does not incur extra communication overhead for the DPs (each iteration of FL requires communications between DPs and server).

In addition, we analyzed the computational and communication overhead of LSFL on edge devices (i.e., DPs) and compared it with similar schemes [37], [52]. The results are shown in Table IV. The computational overhead on the edge devices



(a) MNIST MLP Non-IID (b) MNIST MLP IID

(c) MNIST CNN Non-IID (d) MNIST CNN IID

(e) CIFAR MLP IID (f) CIFAR CNN IID

Fig. 9. Test accuracy of LSFL and FedAvg for different numbers of Byzantine DPs (free-riders) on MNIST dataset (Non-IID and IID) and IID CIFAR dataset.

consists of three components: local model training overhead, noise generation overhead, and local model blinding overhead. Formally, the computational overhead on the edge devices is $\mathcal{O}(T_{tr}) + \mathcal{O}(mT_{ns}) + \mathcal{O}(2mT_{add})$, where m denotes the dimension for uploading intermediate parameters, T_{tr} denotes the time overhead of local model training, and T_{ns} denotes the time overhead of noise generation. Technically, the overhead of noise generation and model blinding overhead is much lower than the overhead of local model training i.e., $T_{add} < T_{ns} \ll T_{tr}$. This indicates that LSFL has a very small effect on edge device computational overhead. This clearly addresses the FL training issue for resource-constrained edge devices. Similarly, DPs' communication overhead is $\mathcal{O}(2m|w|)$, where $|w|$ denotes the communication complexity of a number. Compared with [37], [52], $\mathcal{O}(2m|w|) < \mathcal{O}(m|w_{enc}|)$, where

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

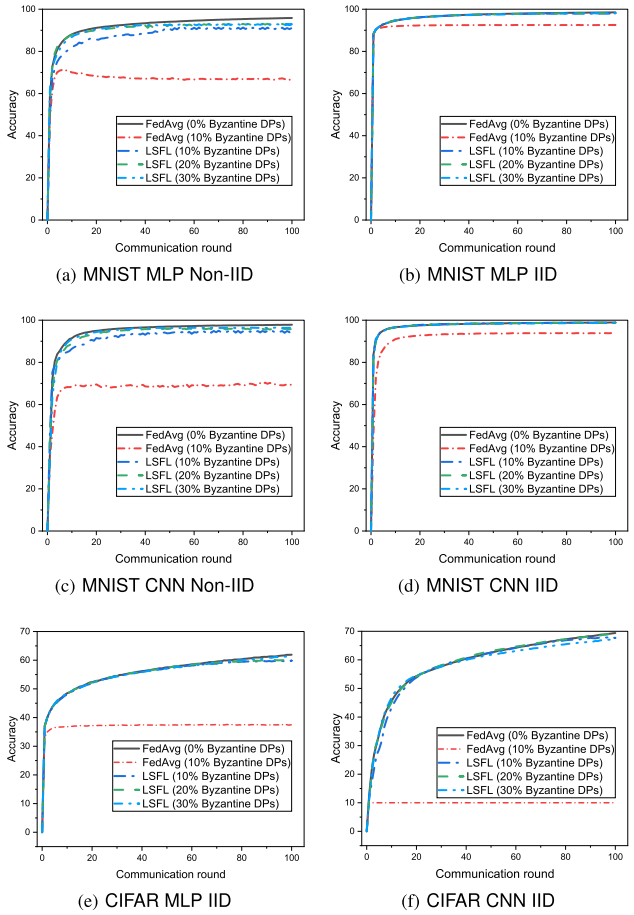


Fig. 10. Test accuracy of LSFL and FedAvg for different numbers of Byzantine DPs (under sign-flipping attack) on MNIST dataset (Non-IID and IID) and IID CIFAR dataset.

$|w_{enc}|$ denotes the communication complexity of an encrypted number. More importantly, since $\mathcal{O}(T_{enc}) \gg \mathcal{O}(T_{ns})$, i.e., the computational complexity of homomorphic encryption is high, $\mathcal{O}(T_{tr}) + \mathcal{O}(mT_{enc}) + \mathcal{O}(mT_{mul}) > \mathcal{O}(T_{tr}) + \mathcal{O}(mT_{enc}) \gg \mathcal{O}(T_{tr}) + \mathcal{O}(mT_{ns}) + \mathcal{O}(2mT_{add})$. As a result, LSFL meets the practical application requirements of this paper.

To investigate the impact of LSFL on FL efficiency further, we tested the training overhead of this scheme. In this experiment, we built a simulation platform in a LAN environment and tested the training latency when the number of participants increased from 20 to 80. The platform used five computers with the same configuration. In particular, we used one computer to simulate SP and TP, and the other four computers to simulate DPs (i.e., participants). Formally, the remaining four computers simulated $\frac{N}{4}$ participants each, where N denotes the total number of participants. In this platform, we used multiple processes to simulate different participants and sockets for communication. The reason for using this simulation for the experiments is the difficulty of finding such a number of computing nodes for the actual experiments. Specifically, we compared and tested the time latency of LSFL and FedAvg to complete one iteration of training. Notably, the experiment was conducted 40 times in order to reduce the effect of exceptional factors, and the

TABLE V
THE PARAMETERS OF EXPERIMENTAL

operating system	Window 10
CPU	Intel(R) Core(TM) i5-9500, 3.00GHZ
memory	16.0 GB RAM
local epochs	3
batch size	32
learning rate	0.01
momentum	0.9
model	CNN
dataset	MNIST

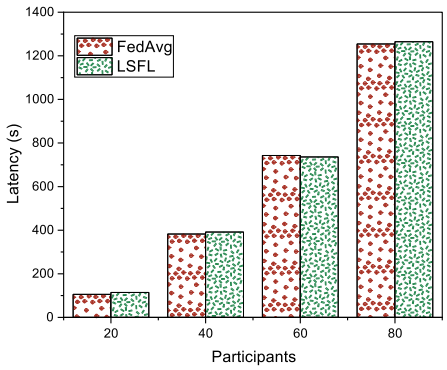


Fig. 11. The time latency of one iteration of LSFL and FedAvg under MNIST CNN IID.

experimental parameters are shown in Table V. The experimental results are shown in Figure 11. From Figure 11, we can observe that compared to the FedAvg, our LSFL only increases the time latency by about 10s. This cost is negligible compared to the overall training. In addition, we also found that the overhead of LSFL was lower than FedAvg in some rounds of training. This is because our LSFL only needs to communicate three times to ensure secure collaborative training. This also further demonstrates that LSFL does not introduce an additional burden in achieving both DPs' privacy preserving and Byzantine robustness. In other words, our scheme achieves the efficiency design goal.

VII. CONCLUSION

In this paper, we proposed a new lightweight privacy-preserving crowdsourced federated learning scheme (LSFL) for edge nodes supporting secure model aggregation and secure Byzantine robustness. This scheme can preserve the data privacy of data providers (DPs) and achieve the same classification accuracy as FedAvg. Specifically, we design a lightweight Two-server secure aggregation protocol for privacy-preserving crowdsourced federated learning, which guarantees the fidelity, security, and efficiency of our LSFL. Experiments show that this scheme can achieve almost the same training accuracy and efficiency as FedAvg without compromising clients' privacy. This demonstrates that our scheme does not sacrifice the performance of the system. Moreover, this scheme does not incur additional computational overhead. Therefore, LSFL is useable and practical for

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

privacy-preserving edge computing federated learning applications. We expect that future work will extend the combination of client privacy preservation and Byzantine robustness, and propose a general framework on which the use of arbitrary Byzantine robustness algorithms can be implemented.

REFERENCES

[1] L. S. Vailshery. (2021). *Internet of Things (IoT)—Statistics and Facts*. [Online]. Available: https://www.statista.com/study/27915/internet-of-things-iot-statista-dossier/

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[3] J. Konecnỳ, H. B. McMahan, X. Y. Felix, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.

[4] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.

[5] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4298–4311, Apr. 2020.

[6] S. R. Pokhrel and J. Choi, "Federated learning with blockchain for autonomous vehicles: Analysis and design challenges," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4734–4746, Aug. 2020.

[7] Y. Zhao et al., "Privacy-preserving blockchain-based federated learning for IoT devices," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1817–1829, Feb. 2021.

[8] C. Zhou, A. Fu, S. Yu, W. Yang, H. Wang, and Y. Zhang, "Privacy-preserving federated learning in fog computing," *IEEE Internet Things J.*, vol. 7, no. 11, pp. 10782–10793, Apr. 2020.

[9] A. Hard et al., "Federated learning for mobile keyboard prediction," 2018, *arXiv:1811.03604*.

[10] J. Wang, L. Wu, H. Wang, K.-K.-R. Choo, and D. He, "An efficient and privacy-preserving outsourced support vector machine training for Internet of Medical Things," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 458–473, Jan. 2021.

[11] Y. Guo, F. Liu, Z. Cai, L. Chen, and N. Xiao, "FEEL: A federated edge learning system for efficient and privacy-preserving mobile healthcare," in *Proc. 49th Int. Conf. Parallel Process. (ICPP)*, Aug. 2020, pp. 1–11.

[12] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 691–706.

[13] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 603–618.

[14] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.

[15] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3454–3469, 2020.

[16] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 5, pp. 1333–1345, May 2018.

[17] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2020.

[18] H. Chaudhari, R. Rachuri, and A. Suresh, "Trident: Efficient 4PC framework for privacy preserving machine learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–26.

[19] S. Sav et al., "POSEIDON: Privacy-preserving federated neural network learning," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–25.

[20] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2201–2218.

[21] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. 29th Int. Conf. Int. Conf. Mach. Learn.*, 2012, pp. 1467–1474.

[22] B. Nelson et al., "Exploiting machine learning to subvert your spam filter," *LEET*, vol. 8, pp. 1–9, Apr. 2008.

[23] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. 23rd Int. Conf. Artif. Intell. Statist.*, in (Proceedings of Machine Learning Research), vol. 108, S. Chiappa and R. Calandra, Eds., Palermo, Italy, Aug. 2020, pp. 2938–2948.

[24] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. 36th Int. Conf. Mach. Learn.*, in (Proceedings of Machine Learning Research), vol. 97, K. Chaudhuri and R. Salakhutdinov, Eds., Long Beach, CA, USA, Jun. 2019, pp. 634–643.

[25] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)* Berkeley, CA, USA: USENIX Association, 2020, pp. 1605–1622.

[26] X. Pan, M. Zhang, D. Wu, Q. Xiao, S. Ji, and Z. Yang, "Justinian's GAAvernor: Robust distributed learning with gradient aggregation agent," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 1641–1658.

[27] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.

[28] Z. Wu, Q. Ling, T. Chen, and G. B. Giannakis, "Federated variance-reduced stochastic gradient descent with robustness to Byzantine attacks," *IEEE Trans. Signal Process.*, vol. 68, pp. 4583–4596, 2020.

[29] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.

[30] J. So, B. Güler, and A. S. Avestimehr, "Byzantine-resilient secure federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2168–2181, Jul. 2020.

[31] Y. Dong, X. Chen, K. Li, D. Wang, and S. Zeng, "FLOD: Oblivious defender for private Byzantine-robust federated learning with dishonest-majority," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2021, pp. 497–518.

[32] M. Hao, H. Li, G. Xu, H. Chen, and T. Zhang, "Efficient, private and robust federated learning," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2021, pp. 45–60.

[33] X. Ma, X. Sun, Y. Wu, Z. Liu, X. Chen, and C. Dong, "Differentially private Byzantine-robust federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3690–3701, Dec. 2022.

[34] I. Giacomelli, S. Jha, M. Joye, C. D. Page, and K. Yoon, "Privacy-preserving ridge regression with only linearly-homomorphic encryption," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.* Cham, Switzerland: Springer, 2018, pp. 243–261.

[35] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, and H. Ludwig, "HybridAlpha: An efficient approach for privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur. (AISec)*, 2019, pp. 13–23.

[36] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1175–1191.

[37] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4574–4588, 2021.

[38] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *Proc. 19th Annu. Int. Conf. Mobile Syst., Appl., Services*, Jun. 2021, pp. 94–108.

[39] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," 2017, *arXiv:1712.07557*.

[40] N. Wu, F. Farokhi, D. Smith, and M. A. Kaafar, "The value of collaboration in convex machine learning with differential privacy," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 304–317.

[41] Y. Li, H. Li, G. Xu, T. Xiang, X. Huang, and R. Lu, "Toward secure and privacy-preserving distributed deep learning in fog-cloud computing," *IEEE Internet Things J.*, vol. 7, no. 12, pp. 11460–11472, Dec. 2020.

[42] S. Truex et al., "A hybrid approach to privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 1–11.

[43] A. Defazio, F. Bach, and S. Lacoste-Julien, "SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives," in *Proc. 27th Adv. Neural Inf. Process. Syst.*, 2014, pp. 1646–1654.

[44] S. Wang et al., "Adaptive federated learning in resource constrained edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 1205–1221, Jun. 2019.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.

[45] X. Wang, Y. Han, C. Wang, Q. Zhao, X. Chen, and M. Chen, "In-edge AI: Intelligentizing mobile edge computing, caching and communication by federated learning," *IEEE Netw.*, vol. 33, no. 5, pp. 156–165, Sep. 2019.
[46] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, and H. Yu, "Federated learning," *Synth. Lectures Artif. Intell. Mach. Learn.*, vol. 13, no. 3, pp. 1–207, 2019.
[47] J. Bernstein, J. Zhao, K. Azizzadenesheli, and A. Anandkumar, "signSGD with majority vote is communication efficient and fault tolerant," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–20.
[48] Y. LeCun, C. Cortes, and C. J. Burges. (2010). *MNIST Handwritten Digit Database. 2010* [Online]. Available: http://yann.lecun.com/exdb/mnist
[49] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," M.S. thesis, Univ. Toronto, Toronto, ON, Canada, 2009.
[50] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 560–569.
[51] L. Lyu, "DP-SIGNSGD: When efficiency meets privacy and robustness," 2021, *arXiv:2105.04808*.
[52] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1639–1654, 2022.



Zhuangzhuang Zhang received the B.S. and M.S. degrees from the College of Information and Computer, Taiyuan University of Technology, Taiyuan, China, in 2017 and 2020, respectively. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Wuhan University, China. His main research interests include AI security and the IoT security.



Libing Wu received the B.S. and M.S. degrees in computer science from Central China Normal University, Wuhan, China, in 1994 and 2001, respectively, and the Ph.D. degree from Wuhan University, Wuhan, in 2006. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He is also a Researcher with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China. His research interests include network security, the Internet of Things, machine learning, and data security.



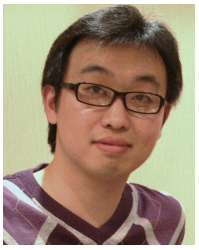
Chuanguo Ma received the B.S. degree from the Changsha University of Science and Technology in 2020. He is currently pursuing the M.S. degree with the School of Cyber Science and Engineering, Wuhan University, China. His main research interests include federated learning and graph neural networks.



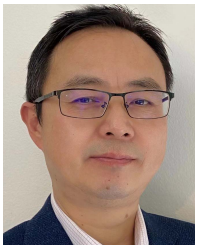
Jianxin Li (Senior Member, IEEE) received the Ph.D. degree in computer science from the Swinburne University of Technology, Australia, in 2009. He is currently an Associate Professor of data science and the Director of the Smart Networks Laboratory, School of Information Technology, Deakin University. His research interests include graph database query processing and optimization, social network analytics and computing, complex network data representation learning traffic, and personalized online learning analytics.



Jing Wang received the Ph.D. degree in computer science from Wuhan University, Wuhan, China, in 2021. She is currently a Lecturer with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan. Her main research interests include cryptography and secure multi-party computation, in particular, secure cloud storage and cryptographic protocols.



Qian Wang (Senior Member, IEEE) received the Ph.D. degree from the Illinois Institute of Technology, USA. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He received the National Science Fund for Excellent Young Scholars of China in 2018. He is also an Expert under National "1000 Young Talents Program" of China. His research interests include AI security, data storage, search and computation outsourcing security and privacy, wireless systems security, big data security and privacy, and applied cryptography. He is a member of the ACM. He was a recipient of the 2018 IEEE TCSC Award for Excellence in Scalable Computing (Early Career Researcher), and the 2016 IEEE Asia-Pacific Outstanding Young Researcher Award. He was a co-recipient of several Best Paper and Best Student Paper Awards from IEEE DSC 2019, IEEE ICDCS 2017, IEEE TrustCom 2016, WAIM 2014, and IEEE ICNP 2011. He serves as an Associate Editor for the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and the IEEE INTERNET OF THINGS JOURNAL.



Shui Yu (Senior Member, IEEE) received the Ph.D. degree from Deakin University, Australia, in 2004. He is currently a Professor with the School of Computer Science, University of Technology Sydney, Australia. His H-index is 65. He has published four monographs and edited two books, more than 400 technical papers, including top journals and top conferences, such as the IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, IEEE/ACM TRANSACTIONS ON NETWORKING, and INFOCOM. He initiated the research field of networking for big data in 2013, and his research outputs have been widely adopted by industrial systems, such as Amazon cloud security. His research interests include big data, security and privacy, networking, and mathematical modeling. He is also serving a number of prestigious editorial boards, including IEEE COMMUNICATIONS SURVEYS AND TUTORIALS (Area Editor), *IEEE Communications Magazine*, and IEEE INTERNET OF THINGS JOURNAL. He served as a Distinguished Lecturer for the IEEE Communications Society (2018–2021). He is a Distinguished Visitor of the IEEE Computer Society, a Voting Member of the IEEE ComSoc Educational Services Board, and an Elected Member of Board of Governor of the IEEE Vehicular Technology Society.

Authorized licensed use limited to: Anhui University. Downloaded on September 14,2026 at 02:35:23 UTC from IEEE Xplore. Restrictions apply.